

[Mastering Sparse CUDA Generation through Pretrained Models and Deep Reinforcement Learning](#)
[Kevin: Multi-Turn RL for Generating CUDA Kernels](#)
[StitchCUDA: An Automated Multi-Agents End-to-End GPU Programing Framework with Rubric-based Agentic Reinforcement Learning](#)
[TOWARDS COLD-START DRAFTING AND CONTINUAL REFINING: A VALUE-DRIVEN MEMORY APPROACH WITH APPLICATION TO NPU KERNEL SYNTHESIS](#)
[GXPO: Group Cross-Lingual Relative Policy Optimization for Code Generation](#)
[GAPO: Robust Advantage Estimation for Real-World Code LLMs](#)
[DRIVE: Data Curation Best Practices for Reinforcement Learning with VErifiable Reward in Competitive Code Generation](#)
[ReVeal: Self-Evolving Code Agents via Reliable Self-Verification](#)
[RL-CODE-INTERPRETER: LLMs REASON WITH CODE VIA SUPERVISED AND MULTI-STAGE REINFORCEMENT LEARNING](#)
[CodeRM-NT: Reward Model for Code RL without Unit Tests](#)
[CODERL+: Improving Code Generation via Reinforcement with Execution Semantics Alignment](#)
[IN-THE-FLOW AGENTIC SYSTEM OPTIMIZATION FOR EFFECTIVE PLANNING AND TOOL USE](#)
[HOW MANY CODE AND TEST CASES ARE ENOUGH? EVALUATING TEST CASES GENERATION FROM A BINARY-MATRIX PERSPECTIVE](#)
[Reinforcement Learning via Self-Distillation](#)
[Risk-Sensitive RL for Alleviating Exploration Dilemmas in Large Language Models](#)
[RiskPO: Risk-based Policy Optimization with Verifiable Reward for LLM Post-Training](#)
[Unlocking Exploration in RLVR: Uncertainty-aware Correctness Advantage Sampling](#)

图片里共有 17 篇论文。它们大致围绕三条主线：

1. 用 LLM + RL 自动生成/优化 GPU、NPU Kernel；
2. 改进代码生成中的奖励、优势估计、多轮 Agent 训练；
3. 解决 RLVR 的稀疏奖励、信用分配、熵坍塌和探索不足。下面逐篇介绍。

一、GPU/NPU Kernel 自动生成

1. Mastering Sparse CUDA Generation through Pretrained Models and Deep Reinforcement Learning

论文提出 SparseRL，研究对象不是普通 CUDA Kernel，而是稀疏矩阵计算 Kernel，尤其是不同稀疏模式下的专用实现。[论文](#) | 代码 核心问题是：稀疏矩阵中非零元素的位置、分布和形状差异极大，很难用一个固定 Kernel 覆盖所有模式；人工为每个稀疏模式写高性能 Kernel 又不现实。SparseRL 的基本流程是：

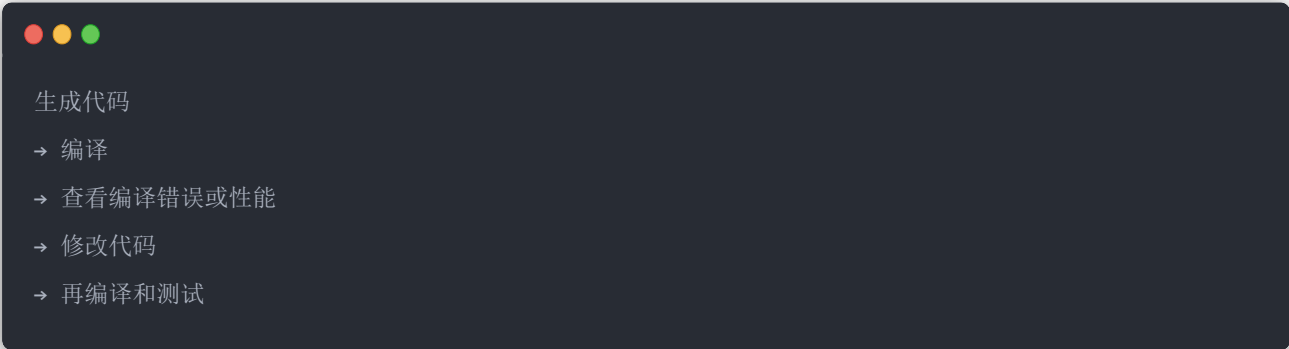
- 输入稀疏矩阵的行列索引及结构信息；
- 预训练模型生成 CUDA 实现；
- 编译并检查正确性；
- 实际运行并测量性能；
- 用正确性和加速比构造奖励；
- 通过 RL 让模型逐渐学会针对稀疏结构生成专用 Kernel。它的重点不是单纯“让 LLM 写 CUDA”，而是把硬件实测结果作为闭环反馈：

Sparse pattern $\rightarrow$ CUDA code $\rightarrow$ compile/run $\rightarrow$ reward $\rightarrow$ policy update.

价值在于证明了 RL 可以让模型学习稀疏访存、线程映射、负载均衡等难以用文字规则完全描述的优化策略。不过，它的任务范围比较专门，不能直接等同于面向任意 PyTorch 算子的 KernelBench 型 CUDA 生成。

2. Kevin: Multi-Turn RL for Generating CUDA Kernels

Kevin，即 “Kernel Devin”，是第一批明确使用多轮 RL 训练 CUDA Kernel 生成模型的工作。[论文](#) 它认为 CUDA 优化天然是多轮过程：



```
生成代码
→ 编译
→ 查看编译错误或性能
→ 修改代码
→ 再编译和测试
```

传统单轮 RL 通常只训练一次生成：

$$x \rightarrow y \rightarrow r,$$

Kevin 则训练完整轨迹：

$$x \rightarrow y_1 \rightarrow f_1 \rightarrow y_2 \rightarrow f_2 \rightarrow \cdots \rightarrow r.$$

这里 f_t 可以是编译错误、正确性测试结果或运行时间。它还需要解决两个问题：

- 长轨迹训练：一个 CUDA Kernel 可能修改很多轮；
 - 跨轮信用分配：最终变快了，究竟是哪一次修改起作用？ Kevin 基于 QwQ-32B 训练，[论文报告](#)：
 - 纯 CUDA Kernel 正确率由 56% 提升到 82%；
 - 相对 PyTorch Eager 的平均加速由 $0.53\times$ 提升至 $1.10\times$ ；
 - 超过论文设置下的 o4-mini；
 - 增加串行修改轮数通常比简单增加并行采样数量更有效。它的核心结论是：CUDA 优化中，“让模型根据真实反馈连续改”比“一次生成很多候选然后挑最好”更有价值。
-

3. StitchCUDA: An Automated Multi-Agents End-to-End GPU Programming Framework with Rubric-based Agentic Reinforcement Learning

StitchCUDA 将研究范围从单个 Kernel 扩展到了端到端 GPU 程序。[论文](#) 系统包含三个主要角色：

- Planner: 设计整体程序结构和优化方案；
- Coder: 逐步实现 CUDA 代码；
- Verifier: 检查正确性，并使用 Nsys、NCU 做性能分析。与 Kevin 相比，它不只是“生成一个 Kernel”，还要处理：
 - Host 侧调度；
 - 多 Kernel 串联；
 - Kernel fusion；
 - cuBLAS epilogue；
 - 内存分配和数据搬运；
 - launch 配置；
- 端到端瓶颈。它提出 rubric-based agentic RL：奖励不仅是最终运行时间，还包含结构化评分项，例如：
 - 是否真的生成自定义 CUDA；
 - 是否正确使用共享内存或融合；
 - 是否消除了不必要的数据搬运；
 - 是否存在硬编码输出；
- 是否只是偷偷调用原始 PyTorch 实现。Rubric reward 与真实执行得到的规则奖励共同用于训练，从而抑制 reward hacking。论文在 KernelBench 上报告接近 100% 的端到端任务成功率，相对多 Agent 基线加速提高 1.72×，相对 RL 模型基线提高 2.73×。它比 Kevin 更偏“完整 GPU 编程 Agent 系统”，Kevin 则更聚焦“多轮 CUDA Kernel RL”。

4. Towards Cold-Start Drafting and Continual Refining: A Value-Driven Memory Approach with Application to NPU Kernel Synthesis

论文提出 EvoKernel，重点解决 NPU 等小众硬件平台缺少训练数据的问题。[论文](#) | [项目页](#)
CUDA 至少有大量公开代码，而 Ascend C 等 NPU 编程生态存在明显的 Data Wall：模型即使很强，也可能不知道基本 API 和优化范式。EvoKernel 不依赖昂贵微调，而是给 Agent 加入可持续积累的经验记忆：

$$M = \{(\text{state}, \text{action}, \text{feedback}, Q)\}.$$

不同阶段使用不同价值函数：

- Cold-start drafting：优先检索能帮助代码编译、通过正确性检查的经验；
- Continual refining：优先检索能降低延迟、改善并行与访存的经验；
- Cross-task sharing：把简单算子中学到的技巧迁移到复杂算子。因此它不是普通 RAG。普通 RAG 根据文本相似度检索；EvoKernel 根据“这段经验过去是否真正提升了当前阶段的目标”进行价值驱动检索。论文在构建的 NPU KernelBench 上将前沿模型正确率从 11% 提升到 83%，并在初始可行实现上取得 3.60× 的中位优化加速。

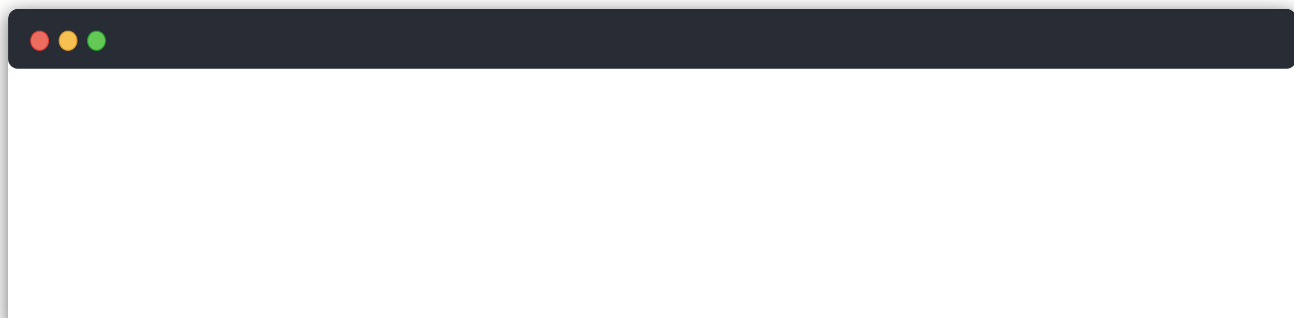
二、代码生成与 Agent RL

5. GXPO: Group Cross-Lingual Relative Policy Optimization for Code Generation

GXPO 研究一个经常被忽略的问题：Code RL 大量使用 Python 训练，结果对 C++、Java、JavaScript、Go 等语言的泛化较差。[论文](#) 普通 GRPO 对同一道题采样一组同语言答案，然后在组内计算相对优势：

$$A_i = \frac{r_i - \mu_r}{\sigma_r}.$$

GXPO 则利用“同一道算法题可以由多种编程语言表达”这一特点，把跨语言解答纳入同一训练结构，使不同语言之间共享算法语义和奖励信息。直观理解：



同一道题

└─ Python 解法

└─ C++ 解法

└─ Java 解法

└─ JavaScript 解法

这些代码表面 token 完全不同，但背后的算法、边界条件和复杂度要求一致。跨语言相对优化扩大了探索空间，也避免模型只记住 Python 特有模板。其主要价值是把多语言代码看作同一语义问题的多种 action representation，而不是多个彼此无关的数据集。论文在 Multilingual LiveCodeBench 等评测上显示，多语言联合训练优于逐语言独立训练和普通 GRPO。

6. GAPO: Robust Advantage Estimation for Real-World Code LLMs

GAPO 解决真实代码编辑任务中 reward 分布偏斜、离群点严重的问题。 [论文](#) | [ACL 2026](#)
| [代码](#) GRPO 使用组平均值作为 baseline:

$$A_i = \frac{r_i - \bar{r}}{s_r}.$$

但在真实代码任务中，一个组的奖励可能是：

$$[0, 0, 0, 0.1, 0.2, 0.2, 0.3, 10].$$

其中一个异常高分会把均值拉高，导致本来不错的样本也得到负优势。GAPO 提出 Group-Adaptive Advantage Denoiser:

1. 在每个 prompt 的奖励组内寻找信噪比最高的区间；
2. 使用该区间的中位数作为自适应 Q ；
3. 用 $r_i - Q$ 代替 $r_i - \bar{r}$ 计算优势。中位数和高信噪比区间对 outlier 更稳健，因此 GAPO 本质上是对 GRPO advantage estimator 的鲁棒化，而不是重新设计奖励模型。它特别适合代码编辑、仓库级修复等奖励并非简单 0/1、容易出现噪声和异常值的环境。

7. DRIVE: Data Curation Best Practices for RLVR in Competitive Code Generation

DRIVE 的重点不是提出新的 RL loss，而是回答：竞赛代码 RL 到底应该使用什么训练数据、怎样采样？[论文](#) | [代码](#) 训练分为三部分。第一阶段是 SFT：

- 基于强模型蒸馏代码推理数据；
- 将题目划分为 easy、medium、hard；
- 提高困难样本在 SFT 中的比例；
- 混入通用代码和推理数据。第二阶段是大规模 GRPO：
- 使用较大且难度分布较均匀的数据；
- 每题约 8 个 rollout；
- 相对较短的生成窗口；
- 目标是扩大策略熵、减少重复和截断。第三阶段称为 Pre-GRPO：
- 使用规模较小但质量高的困难题；
- 每题扩大到约 64 个 rollout；
- 持续保留模型当前最难的题目；
- 形成 hard-focus curriculum。它的重要结论是：

Code RL 的效果高度取决于 prompt 难度、rollout 数量和课程安排，未必主要取决于 PPO、GRPO 或其他优化器之间的小改动。

8. ReVeal: Self-Evolving Code Agents via Reliable Self-Verification

ReVeal 认为生成模型与验证模型之间存在不对称：模型可能会写代码，但不一定能可靠判断自己的代码是否正确。[论文](#) 它把代码推理组织成多轮循环：

generate code → generate tests/verify → execute → revise.

核心创新包括：

- 显式训练 self-verification，而不是把验证当成自然涌现能力；
- 使用真实执行工具提供可靠反馈；
- 通过 TAPO 做 turn-level credit assignment；
- 让代码生成与测试生成共同演进。比较有意思的是，模型训练时只使用约 3 轮交互，但推理时可以稳定扩展到 20 轮以上，说明模型学到了一定的“持续改进循环”，而不只

是背诵三轮模板。它与 Kevin 都是多轮 RL，但区别是：

- Kevin：重点是 CUDA 性能优化；
- ReVeal：重点是普通代码生成中的自验证、测试生成和长程修正。

9. R1-Code-Interpreter: LLMs Reason with Code via Supervised and Multi-stage Reinforcement Learning

R1-Code-Interpreter 训练模型在推理过程中自主调用代码解释器，而不是只在最终输出代码。[论文](#) | [代码](#) 模型可以执行多轮：



```
文字推理
→ 编写 Python
→ 执行
→ 阅读输出
→ 修正推理
→ 再次执行
```

困难在于不同任务从 Code Interpreter 获得的收益差异很大。如果把所有任务混在一起直接做 RL，大量“不需要工具”或“工具暂时帮不上忙”的样本会稀释训练信号。论文根据 measured improvement potential 划分训练阶段：

- 先训练使用代码后提升最明显的样本；
- 再逐渐加入提升空间较小的样本；
- 构成多阶段 curriculum RL。在 Qwen2.5 3B/7B/14B 上，这种课程将平均 RL 增益从约 3.4% 提升到 9.3%。最终 R1-CI-14B 在 37 个测试任务上的平均准确率由 44.1% 提升到 72.4%。

10. CodeRM-NT: Reward Model for Code RL without Unit Tests

CodeRM-NT 研究没有可靠单元测试时怎样进行 Code RL。[论文](#) | 代码 传统代码 RL 依赖：

$$r = \mathbf{1}[\text{all unit tests pass}].$$

但大量真实代码任务没有完备测试，而且 LLM 自动生成的测试也可能错误或覆盖不足。

CodeRM-NT 的数据构造流程是：

1. 用 LLM 引导的 MCTS 探索不同代码候选；
2. 执行代码并收集 execution trace；
3. 让判别过程结合代码、轨迹和执行状态产生奖励标注；
4. 用这些标注训练不依赖测试用例的 Code Reward Model；
5. 在 RL 时由 CodeRM-NT 为生成代码打分。该奖励模型还可估计题目难度，用于 curriculum learning。实验中，其奖励信号优于合成单元测试奖励。但它也有一个基本风险：把完全可验证的真实执行奖励换成了学习式奖励模型后，必须警惕 reward hacking 和跨分布误判。

11. CodeRL+: Improving Code Generation via Reinforcement with Execution Semantics Alignment

CodeRL+ 解决二元 pass/fail 奖励过于稀疏的问题。[论文](#) | [ACL 2026](#) | [代码](#) 普通 RLVR 只知道程序是否通过：

```
答案 A: 第 3 行就写错了 → 0
答案 B: 只差一个边界条件 → 0
```

两者奖励相同，但学习价值显然不同。CodeRL+ 让模型额外推断变量级 execution trajectory，例如：

```
i=0, sum=0
i=1, sum=3
i=2, sum=7
...
```

从而建立代码 token 与运行语义之间的对应关系。训练时并行优化：

$$L = L_{\text{code-RLVR}} + \lambda L_{\text{execution-semantics}}$$

尤其重要的是，失败 rollout 不再完全浪费：即使代码最终错误，其中一部分变量传播和中间状态仍可以提供语义监督。论文报告代码生成 pass@1 相对提升约 4.6%，并在代码推理、输出预测任务上表现出迁移收益。

12. In-the-Flow Agentic System Optimization for Effective Planning and Tool Use

论文提出 AgentFlow 和 Flow-GRPO，研究怎样在 Agent 实际多轮运行过程中训练 Planner。 [论文](#) | [项目页](#) | [代码](#) AgentFlow 包含：

- Planner;
- Executor;
- Verifier;
- Generator;
- 持续更新的共享 memory。很多 Agent 系统要么完全不训练，要么离线收集轨迹后单独训练，与实际运行时的状态分布脱节。AgentFlow 则把 Planner 放在真实多轮 loop 内做 on-policy 优化。Flow-GRPO 将长轨迹拆成一系列单轮策略更新，同时把最终轨迹奖励广播到各轮：

$$A_t \approx \frac{R(\tau) - \mu_R}{\sigma_R}.$$

它近似解决“最终成功应该归功于哪一步规划”的问题。优点是简单、稳定；缺点是所有 turn 接收相同 outcome 信号，信用分配仍比较粗。它不是专门的 Code RL 论文，但其 Planner-Executor-Verifier 结构可以直接迁移到代码修复、CUDA 生成和自动调优 Agent。

三、测试、奖励密化与探索

13. How Many Code and Test Cases Are Enough? Evaluating Test Case Generation from a Binary-Matrix Perspective

这篇论文研究如何科学评测 LLM 生成的测试用例。 [论文](#) | [数据集](#) 作者构造一个二进制矩阵：

$$M_{ij} = \begin{cases} 1, & \text{测试 } j \text{ 能检测出错误代码 } i \\ 0, & \text{否则.} \end{cases}$$

其中：

- 行代表错误代码；
- 列代表测试用例；
- 一行就是某个错误程序的 failure signature。矩阵的秩表示独立错误模式的数量，并给出区分这些错误模式所需测试数量的紧上界。作者进一步提出 WrongSelect，从海量错误提交中选择规模较小但内部差异最大的错误代码集合，并构造 TC-Bench。它揭示了传统评测的 score inflation：如果 benchmark 中充满相似的简单错误，生成几个普通测试就能得到很高分；而在覆盖罕见错误模式的 TC-Bench 上，先进测试生成方法也只有约 60% 的错误排除率。这篇论文不直接训练代码模型，但对 CodeRL+、ReVeal、CodeRM-NT 等依赖测试或验证信号的方法非常关键。

14. Reinforcement Learning via Self-Distillation

论文提出 Self-Distillation Policy Optimization，简称 SDPO。 [论文](#) | [代码](#) 普通 RLVR 只利用标量结果：

$$(x, y) \rightarrow r.$$

但代码环境经常还能返回编译错误、运行时错误、judge feedback。SDPO 构造两种策略：

- Student: $\pi_{\theta}(y \mid x)$
- Self-teacher: $\pi_{\theta}(y \mid x, f)$ 其中 f 是环境反馈。因为 teacher 看到了错误信息，其 next-token distribution 往往更清楚应该怎样修正。SDPO 再把这一分布蒸馏回不带反馈的策略：

$$L_{\text{SDPO}} = D(\pi_{\theta}(\cdot \mid x, f) \parallel \pi_{\theta}(\cdot \mid x)).$$

它不是传统“强教师蒸馏给弱学生”，教师和学生是同一个模型，只是教师拥有额外反馈上下文。与 GRPO 的本质差异是：

- GRPO：用最终标量奖励调整整条序列概率；
- SDPO：把文本反馈转化为逐 token 的分布监督；
- GRPO 信号稀疏，SDPO 信号更密；
- SDPO 不需要独立教师或显式 reward model。

15. Risk-Sensitive RL for Alleviating Exploration Dilemmas in Large Language Models

论文提出 RS-GRPO，目标是改善 RLVR 只提高 pass@1、却损害 pass@k 和解法多样性的问题。[论文](#) | [代码](#) 普通 RL 近似优化平均奖励：

$$J_{\text{mean}}(\pi) = \mathbb{E}[R].$$

这容易让模型强化已有的高概率解法，策略越来越尖锐，最后发生熵下降。RS-GRPO 引入风险寻求目标，在平均奖励与高奖励尾部之间插值。直观上，它更重视：

- 当前成功率不高；
- 但偶尔能够发现正确新路径；的困难 prompt。这里 “risk-seeking” 不是鼓励错误，而是允许策略为低概率、高潜力解法保留更多概率质量。实验表明，它在数学推理上提高 pass@k，同时基本保持或提高 pass@1。该方法改动较小，适合直接加入 VeRL/GRPO 训练代码。

16. RiskPO: Risk-based Policy Optimization with Verifiable Reward for LLM Post-Training

RiskPO 同样从风险度量角度重新设计 RLVR，但比 RS-GRPO 使用了更系统的 Mixed Value-at-Risk。[论文](#) 普通方法只看奖励均值，而 Mixed VaR 对奖励分布的多个区域加权：

$$\text{MVaR}(R) = \sum_k w_k \text{VaR}_{\alpha_k}(R).$$

这样既能关注困难样本和稀有成功轨迹，又不会完全被单个极端 rollout 支配。论文还引入 question bundling：把多个问题组成 bundle 后共同估计风险和更新信号，从而缓解单题只有 0/1 奖励造成的高方差。与上一项 RS-GRPO 的区别可以粗略概括为：

- RS-GRPO：在均值和最大/高回报方向之间做风险寻求，重点是突破局部最优；
- RiskPO：用多分位点 Mixed VaR 重新刻画整个奖励分布，并配合 question bundling；
- 两者都试图缓解 GRPO 对高概率已有路径的过度强化和熵坍塌。

17. Unlocking Exploration in RLVR: Uncertainty-aware Advantage Shaping for Deeper Reasoning

论文提出 UCAS，即 UnCertainty-aware Advantage Shaping。 [论文](#) | [ACL 2026](#) 它指出 GRPO 把同一个序列优势广播给所有 token：

$$A_{i,t} = A_i, \quad \forall t.$$

但推理链中不同 token 的作用不同：

- 有些只是格式、连接词；
- 有些是高不确定性的关键决策；
- 有些是模型非常自信但实际上错误的 token。UCAS 分为两级。第一层是 response-level confidence modulation：根据整条回答的内部置信度调整优势，重点奖励“不确定但最终正确”的探索轨迹。第二层是 token-level asymmetric certainty penalty：
- 正确且不确定的关键 token：鼓励；
- 错误且过度自信的 token：重点惩罚；
- 普通低影响 token：降低权重。因此 UCAS 不需要额外 PRM 或 reward model，而是直接利用策略模型自身的 logits 构造细粒度优势。实验显示它能提升推理多样性、恢复策略熵，并在 1.5B 和 7B 模型的多个数学基准上优于强 RLVR 基线。

四、把 17 篇放在一起看

方向	论文	主要解决的问题
专用 Kernel RL	SparseRL	针对稀疏结构生成专用 CUDA
多轮 Kernel RL	Kevin	根据编译和性能反馈连续优化 CUDA
端到端 GPU Agent	StitchCUDA	同时优化 Kernel、Host 和系统配置

方向	论文	主要解决的问题
小众硬件冷启动	EvoKernel	用价值记忆解决 NPU 代码数据匮乏
多语言代码 RL	GXPO	Python 训练向其他语言泛化不足
鲁棒优势估计	GAPO	奖励偏斜和异常值污染 GRPO advantage
数据课程	DRIVE	Code RL 应选择什么题、多少 rollout
多轮自验证	ReVeal	让代码生成与测试验证共同演进
工具使用	R1-Code-Interpreter	学会多轮调用代码解释器
无测试奖励	CodeRM-NT	没有单元测试时如何构造代码奖励
执行语义	CodeRL+	将 pass/fail 密化为变量级运行语义
Agent Planner RL	AgentFlow	在真实多轮工具流内训练规划器
测试评测	TC-Bench	怎样选择最有诊断能力的错误代码和测试
反馈自蒸馏	SDPO	将编译错误等文本反馈变成 token 监督
风险寻求探索	RS-GRPO	提高 pass@k 和新解法探索
风险分布优化	RiskPO	用 Mixed VaR 缓解熵坍塌
不确定性塑形	UCAS	根据 token 置信度做细粒度信用分配

最值得关注的整体趋势是：研究重点已经从“换一个 PPO/GRPO loss”转向四件更实际的事情——真实执行反馈、多轮交互、细粒度信用分配，以及训练数据与 rollout 的选择。对于你更关注的 CUDA/Kernel 自动生成方向，建议优先深入阅读顺序是：

Kevin → StitchCUDA → SparseRL → EvoKernel → SDPO/CodeRL+.

前四篇告诉你系统怎样搭；后两篇则提供了把编译错误、执行轨迹和性能反馈进一步密化成训练信号的思路。